

1 Learning Outcomes

This assignment will require students to:

1. Apply specific search algorithms studied in the course to unfamiliar problems.
2. Select appropriate data structures to optimize the performance of the algorithms.
3. Use algorithm analysis techniques to determine the asymptotic complexity of algorithms.

2 General Instructions

1. **Programming Language:** All students are required to use the **Java programming language** for this assignment. You may only use in-built Java library methods and classes (that are listed in the documentation for the Java Standard Edition API), as well as ones you write yourself. You are **not permitted** to import any third-party libraries to use in this assignment.
2. **Individual Assignment:** Each student must complete the main tasks specified as an individual to demonstrate achievement of the learning outcomes; no collaboration is permitted.
3. **Marking Guideline:** Marks are provided for functionality and for efficiency (optimising the time complexity of your algorithms). The purpose of these tasks is three-fold: (i) you need to problem-solve: figure out how best to solve each task, (ii) implement and test the solution, and (iii) determine the complexity of the algorithms used in solving the problems.
4. **Academic Integrity:** Each student must complete the tasks in this section individually, no collaboration with other students is permitted. All code must be your own original work. The use of generative AI tools, including but not limited to: ChatGPT, Copilot, Tabnine, OpenAI Codec, etc, is NOT permitted. Any collaboration with other students, or use of generative AI will be considered a breach of the Academic Integrity Policy and may be referred to the Student Academic Conduct Officer (SACO) for disciplinary action.

3 Getting Started

Download the package `assignment1base.zip` from Canvas. In this package you will find three files:

1. `TrafficAssistant.jar`: A pre-compiled Java file that contains two classes: `MapGenerator` and `TrafficCheck`. These classes are black-boxes, you do not need to know how they work.
 - When compiling, you will need to ensure this `jar` file is on the Java classpath. You can use the `-cp` flag when invoking the Java compiler to set the classpath.
 - This file was compiled with Java 21. To run it, you will need to use version 21 or higher.
2. `TrafficAnalyser.java`: This is where you will complete your assignment, some starting code has been provided to allow compilation.
3. `TrafficMain.java` This is the main class to control the program, calling your methods from `TrafficAnalyser.java`. You may modify this file underneath the marked comment in order to test your code, but the modifications will not be really not part of the assignment.

4 Problem Scenario

In the bustling metropolis of Veridian City, chaos reigns during rush hour. The streets overflow with vehicles, commuters jostling for space, and traffic signals blinking in desperate confusion. The city council has had enough. They have turned to you, a brilliant second-year algorithms student—to untangle this vehicular web. Your mission is to step into the role of one of the city’s digital traffic engineers, analyse vast streams of traffic data, identify bottlenecks, optimize signal timings, and propose innovative solutions. The city’s future hangs in the balance, and every algorithm you design could mean smoother commutes, reduced emissions, and happier citizens.

Figure 1 shows an example of a roadmap of a city like the Veridian city. In fact it is the test map given in `TrafficMain.java` in the base code supplied to you. **Note that in every run of your program, a map generated randomly by the `MapGenerator` class will be used.**

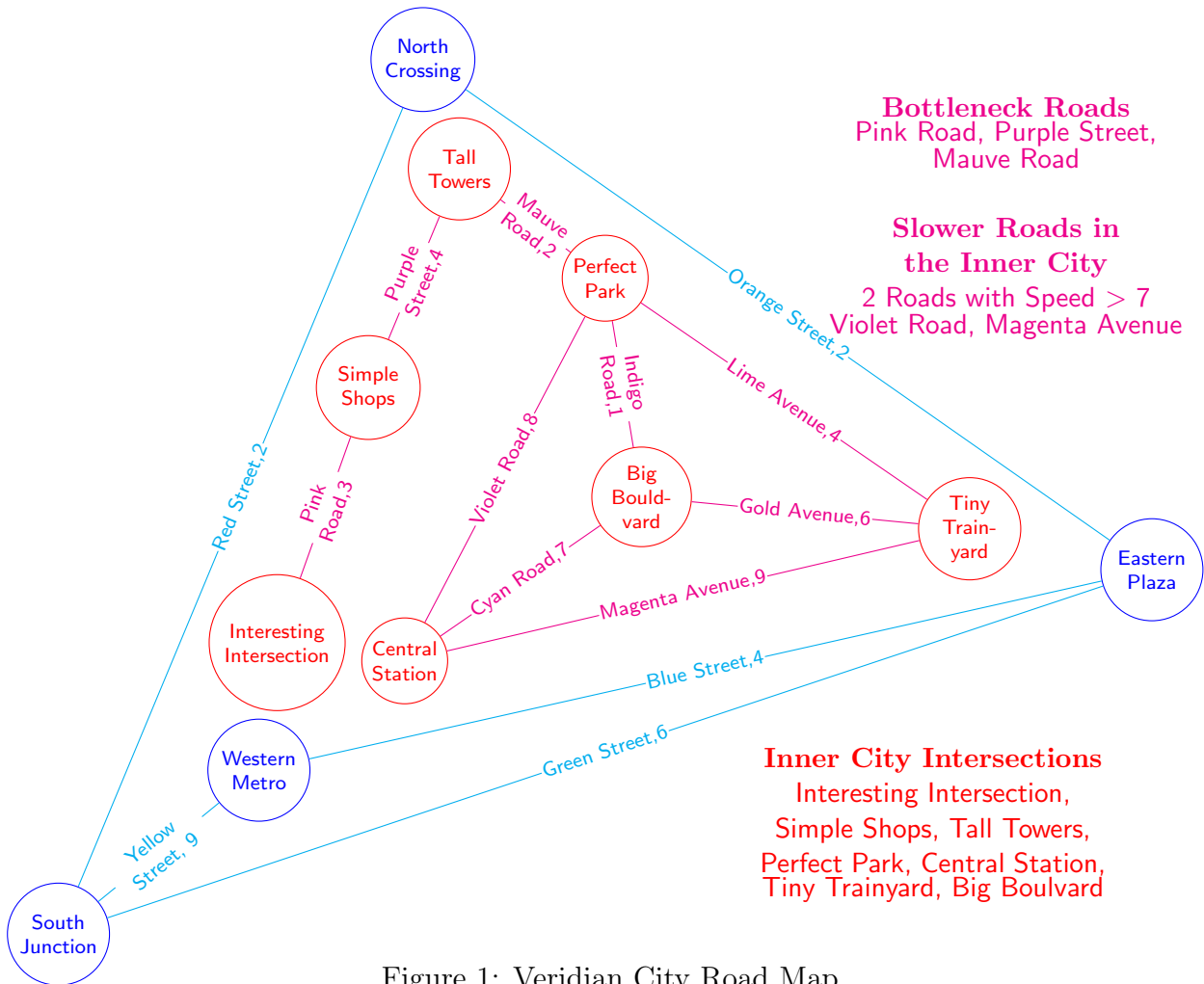


Figure 1: Veridian City Road Map

5 Problem Tasks

There are 4 tasks in this assignment. You have to complete all of the tasks to get total 100 marks.

5.1 Divided City (25 Marks)

In Veridian City, the rapid urban expansion has led to a chaotic network of roads. Multiple construction companies have been building roads independently, leading to a disjointed city layout. Some residents have even reported that they are unable to reach the ‘inner city’ from their suburbs due to the lack of connected roads. For example, see the ‘inner city’ intersections in Figure 1. Your first task is to analyse the city’s road network and determine the extent of this problem.

5.1.1 City Mapping (15 Marks)

Take the list of roads built by various construction companies in Veridian City, and create some order out of the chaos in preparation for future processing.

- Write the method `void loadMap()`. The first few lines in this method are written for you. It will call the city registry to get a complete list of roads in the city.
- This list will be a serialized string in the following format:
`"{{road1}}, {road2}}, {road3}}, ... {roadN}}"`.
- Each road will be in the following format. The endpoints in the format are the intersection names and the travel times for the roads are in minutes.
`"{road-name, first-endpoint, second-endpoint, average-travel-time}"`
- For example, a city with three roads might look like this:

```
"{{Alpha Road, Alpha/King Lights, Alpha/Park Crossing, 3.5},
  {King Street, King/Park Roundabout, Alpha/King Lights, 4},
  {Park Close, Alpha/Park Crossing, King/Park Roundabout, 8.2}}"
```
- This method has no return value, but you may assume for marking purposes that it will be called first, before any other methods.
- Note the following information regarding the road list:
 - For convenience, the intersections where roads meet are named after the most prominent point of interest at that intersection.
 - You will not receive conflicting or erroneous information, that is, road names will not be repeated on the list provided, travel times will not be negative numbers, etc; so error checking is not required.
 - The number of roads is randomised, as are the road names, intersection names, and order of the roads. To help with testing, a seed value has been exposed in `TrafficMain.java` that will be passed to the map generator. By setting the same seed value, you will be able to generate the same map across multiple runs.
 - Veridian City currently has no one-way roads, though this may change in the future. If a road exists, it may be travelled in both directions.
- **Hint:** This method is arguably the most important method in the program. The way you setup the data structures here will affect the performance of all other methods in the program. Notice that the street and intersection names are all strings but strings are usually not suitable for a graph representation. So you should think of mapping the strings to integers. There could be indexes for the intersections and/or streets. You may consider storing the strings in a list and then use the indexes. You will then need to get the string from an index or vice versa. To find the index for a string, you may perform some search: linear, binary, interpolation, or hash table search. To find the string for an index, usually an array is useful. Carefully design these.

5.1.2 Critical Disconnection (10 Marks)

Some residents of Veridian City have been complaining that they cannot reach the ‘inner city’ from their homes. For example, in Figure 1, we cannot go to the inner-city intersections from South Junction, North Crossing, Western Metro, or Eastern Plaza. You will need to determine the extent of this problem to aid in planning future road construction.

- The ‘inner city’ is defined as follows: If there are any intersections that cannot be reached from other intersections, then it is possible to group intersections, where every intersection in a group can reach any other intersection in the same group through a series of roads. **Given this, the ‘inner city’ shall be defined as the largest such group of intersections.**
- Write boolean `isInInnerCity(String intersectionName)` method. The input will be the name of an intersection in the city, and you must determine whether that intersection is a part of the ‘inner city’. The answer will be a boolean value: true if the provided intersection is a part of the largest group, and false if it is not. **Use the disjoint sets to solve this problem.**
- **Hint:** Two nodes are in the same set if they can be reached from one another. Consider union with ranks and find with path compression for performance improvement.

5.2 Tangled Web (25 Marks)

In the heart of Veridian City, the roads weave a complex web of connections. As a digital traffic engineer, you must navigate this maze, identify slower roads, and untangle busier intersections.

5.2.1 Go, Slow, Stop (10 Marks)

The city’s traffic problem is exacerbated by roads that are notorious for their slow travel times. Identifying these roads is crucial for future traffic improvement plans. In Figure 1, see the slower roads in the inner city with the speeds larger than a given threshold.

- Write the method `int countInnerCitySlowRoads(double threshold)`. The input will be a threshold that will be a positive rational number, and you must return the number of roads in the ‘inner city’ with average travel times that are strictly worse than this threshold.
- **Hint:** At first glance, you may think to check every road. With some clever thought, and the right data structures setup while loading the map, you may be able to solve this one much faster, with the full efficiency bonus scored if you can achieve $O(\ln n)$.

5.2.2 Uncorking the Bottle (15 Marks)

Some roads in the city act as bottlenecks, disrupting the smooth flow of traffic. Identifying these roads will help in planning road expansions or redesigns. See the examples of bottleneck roads in Figure 1. **Note bottleneck roads could be in the entire city, not just in the ‘inner city’.**

- Write the method `String[] cityBottleneckRoads()`. This method needs to return an array of road names in the ‘inner city’ such that, if any single road in the array were to be closed, one or more intersections in the ‘inner city’ would no longer be part of that group. That is, removing one of the roads in the array would disconnect one or more intersections in the ‘inner city’. **Your solution must use an iterative depth-first approach.**
- **Hint:** There are pseudocode of various algorithms in the lecture slides and also many example programs on the Canvas site. Consider carefully which one would be the most appropriate base algorithm, and then modify it to suit the task.

5.3 Ministerial Visit (30 Marks)

The Minister for Transport is coming to visit Veridian City. However, the visit comes with its own set of challenges. As the city’s digital traffic engineer, you have to address them.

5.3.1 The Minister's Speech (12 Marks)

The minister will be giving a speech at a certain intersection in the city. Ensuring the Minister's safety during the speech is paramount, and the security services have asked for your help in this task. Consider the road map in Figure 2. If the minister will be speaking at the intersection marked, and the number of hops is set to 1, then the roads with a red line across them would need to be closed.

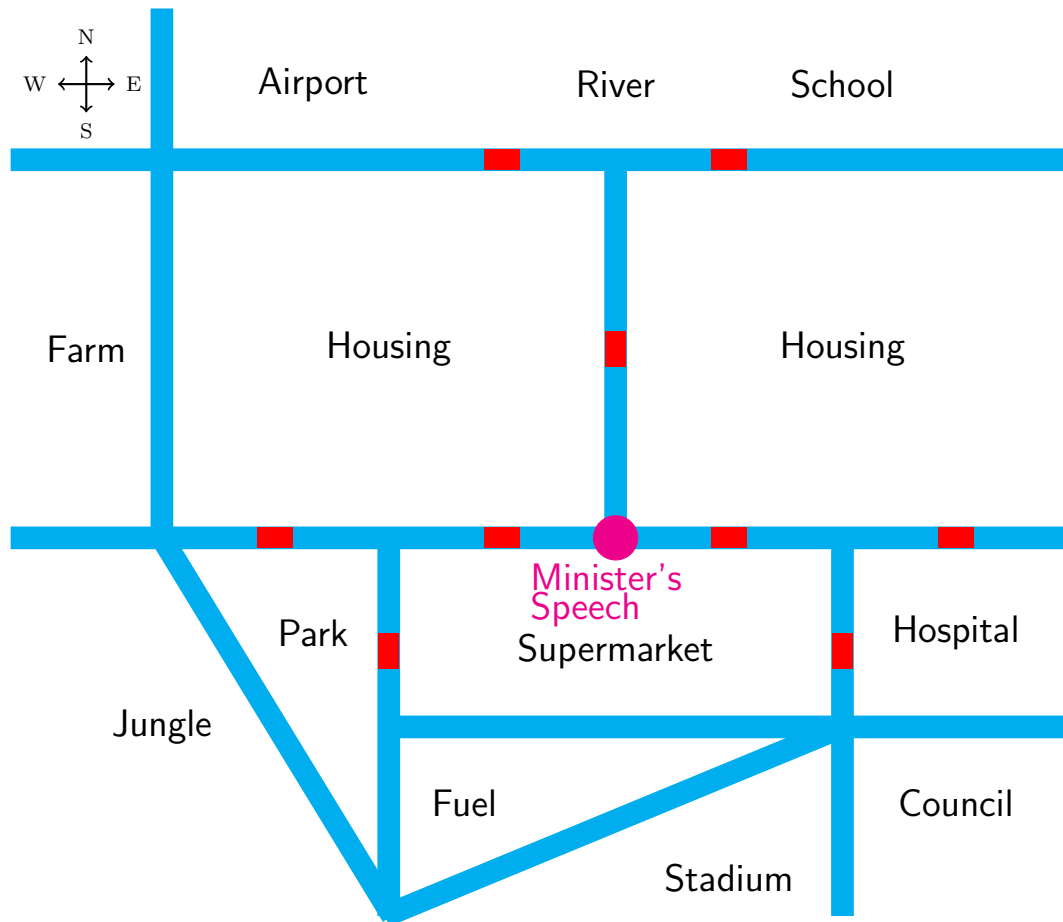


Figure 2: For the minister's speech, lock down each intersection within the number of hops 1

- Write the method `String[] lockdownIntersection(String intersectionName, int hops)`. This method takes in the name of the intersection where the minister will be giving the speech from, and a number of hops to define the area for lockdown. The method then returns an array of roads that need to be closed during the minister's speech to prevent anyone from accessing an intersection from which the intersection the minister will be speaking from could be reached by traveling a number of roads less than or equal to the number of hops provided. **Your solution must use an iterative breadth-first approach.**
- **Hint:** There are pseudocode of various algorithms in the lecture slides and also many example programs on the Canvas site. Consider carefully which one would be the most appropriate base algorithm, and then modify it to suit the task.

5.3.2 I protest! (18 Marks)

We have an emergency! The city's traffic problems have led to protests during the minister's visit. Your task is to create the best action plan to contain the protests by closing roads within the city.

Consider the road map in Figure 3. The protests initially start at the three intersections marked. However, we will consider two groups of protests here. The two north-most protests are considered the same group as they are connected by a single unclosed road. The south protest is not connected

to another other protest by a single unclosed road. We see that the northern protest group can spread to four more intersections while the southern group can spread to three intersections. As $4 > 3$, we will choose to isolate the northern group first by closing the four roads marked.

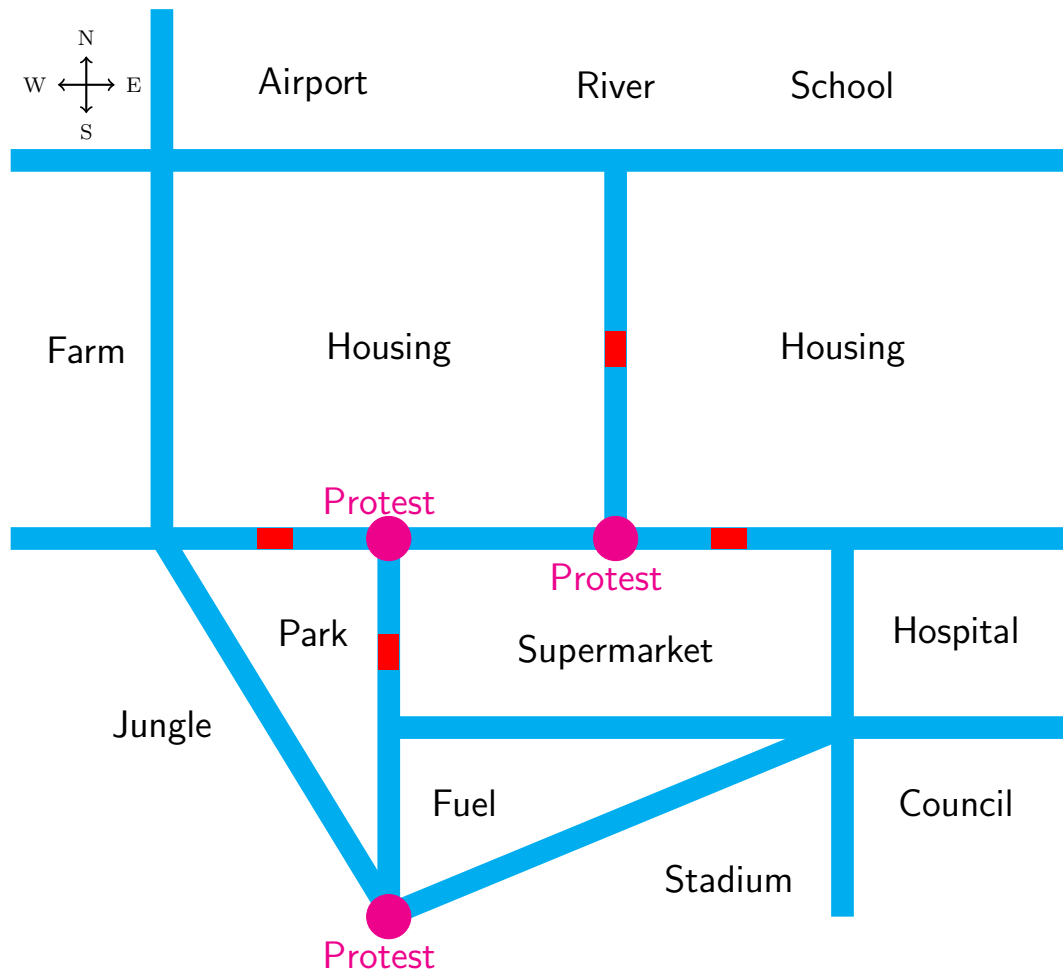


Figure 3: For the minister's speech, lock down an intersection with the number of hops 1

Following the above road closure, the northern group is completely closed now, but the southern group can move. The protest will spread to the three adjacent intersections but stay as one group and the only group at this stage. To contain this group completely, we will need to close five roads. See Figure 4 to see the example after the second round of road closure. With all protests closed from spreading further, we finish by closing the total number of nine roads.

- Write the method `int containProtests(String[] intersectionNames)`. This method takes in an array of intersections where protesters are initially gathered, and must return the minimum number of roads that need to be closed to contain the spread of the protests.
- As you see from the above example, the process runs as a simulation, where, on each time step, the city performs a road closure action, then the protests spread by one step. On the city's turn, roads should be closed around a single connected group of intersections with protesters that will spread to the largest number of intersections on their turn (ties may be broken by selecting the group that will spread the protest down the fewest number of roads). On the protests turn, they will spread down any adjacent non-closed roads to connected intersections. The protests will only travel a single 'hop' on their turn, then the city may act again.
- **Hint:** This problem seems very complex, but once the simulation steps are understood, the implementation is quite straight-forward. At least, it is if you have the right data structures. . . Start by breaking down the problem, the implementation should involve these steps: 1. Find

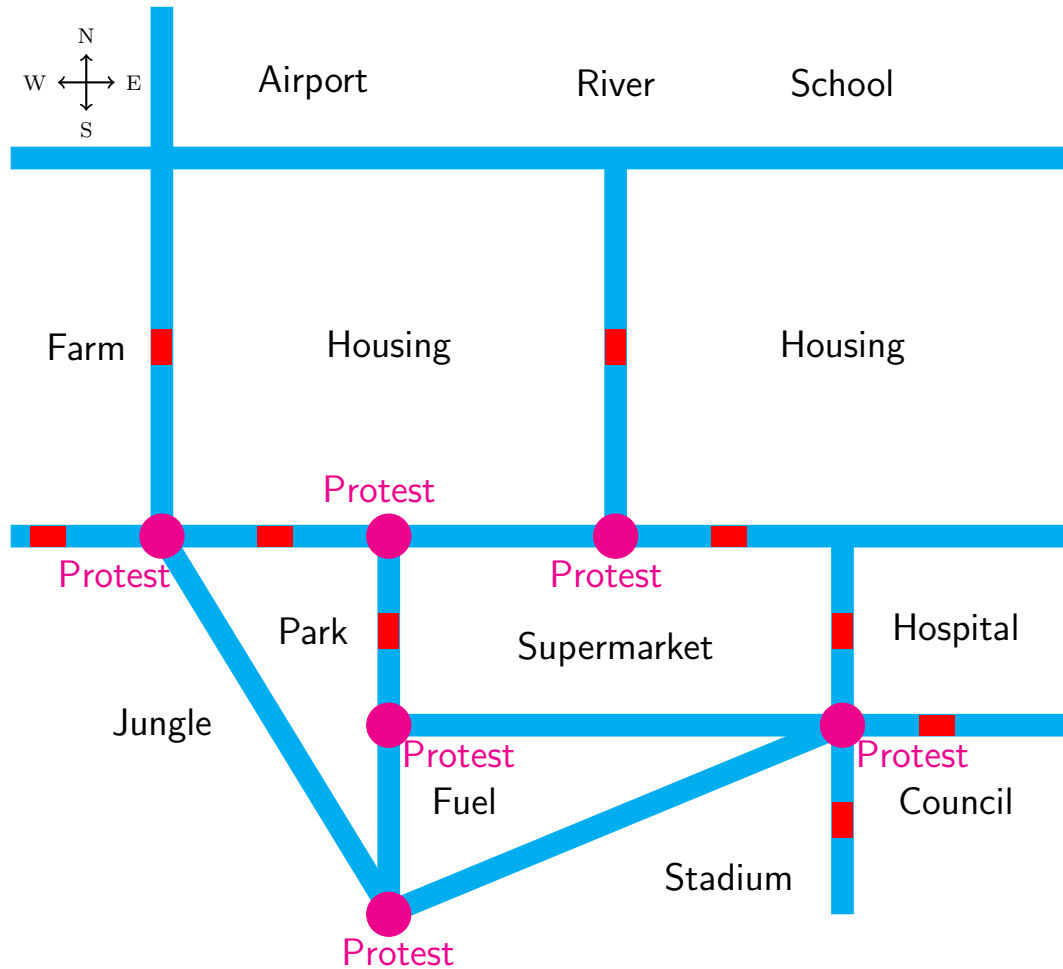


Figure 4: For the minister’s speech, lock down an intersection with the number of hops 1

all ‘protest regions’ (connected intersections with protesters – note that two intersections are not connected if the road between them is closed), additionally for each region keep track of its frontier (neighbouring peaceful intersections the protest would spread to on the next step), and the outbound roads of that frontier. 2. Isolate the most dangerous region (the one that would spread to the most peaceful intersections on the next step), adding its outbound roads to the answer and closing them for the next step. 3. Spread the protest in the remaining regions outward by one non-closed road (this may result in two or more protest regions reaching each other and becoming a single connected protest region, you will need to perform a merge if this happens). For each step, consider what would be the best algorithm to use, and whether one or more of the search algorithms discussed in class would be helpful for structuring the simulation.

5.4 Part 4: Report (20 marks)

Your solution should be accompanied by a short report. This report should explain how you have implemented each algorithm, and demonstrate the time complexity for each one. You must prove the time complexity stated in your report.

6 Submission

The deliverable for this assignment is a zip file named `A1cxxxxxxx` (where `xxxxxxx` is your student number). This zip file should contain only the following files:

- `TrafficAnalyser.java`: This file should contain your solutions to each task, implemented according to the specifications provided

- `readme.txt`: Contains your name and student number, along with any special notes about running your program for marking.
- `report.pdf`: A short report containing your analysis of each algorithm, including time complexity for your implementations with proofs.
- `AssessmentCoverSheet.pdf`: A completed Assessment Item Cover Sheet using the pdf found from the link below. If the cover sheet is not included, your assignment cannot be marked.

https://www.newcastle.edu.au/__data/assets/pdf_file/0008/75383/AssessmentItemCoverSheet.pdf

You do not need to include the file `TrafficMain.java` as this will be replaced with a fresh copy before marking. For this reason, while you modified `TrafficMain.java` to assist with development and debugging, for marking, your program MUST run with the original version of this file. Notice that a sample map with test cases has been provided in `TrafficMain.java` to help you get started. Each test case shows the input and expected output of the corresponding method.

7 Marking Criteria

Total: 100 marks

7.1 Task 1: 25 marks

1. `loadMap`: 15 marks

- Correctly stores road layout into one or more data structures [10 marks]
- Efficient data structure(s) used [5 marks]

2. `isInInnerCity`: 10 marks

- Correct implementation to identify intersections not in ‘inner city’ [6 marks]
- Efficient implementation in time complexity [4 marks]

7.2 Task 2: 25 marks

1. `countInnerCitySlowRoads`: 10 marks

- Correct implementation to count roads in the ‘inner city’ with a travel time greater than the threshold provided [7 marks]
- Efficient implementation in time complexity [3 marks]

2. `cityBottleneckRoads`: 15 marks

- Correct implementation using depth-first search from generic algorithm to find and return roads that would disconnect the ‘inner city’ if removed [10 marks]
- Efficient implementation in time complexity [5 marks]

7.3 Task 3: 30 marks

1. `lockdownIntersection`: 12 marks

- Correct implementation of breadth-first search from generic algorithm returning correct solution [8 marks]
- Efficient implementation in time complexity [4 marks]

2. containProtests: 18 marks

- Correct simulation of spreading protests, calculates correct number of roads to close [12 marks]
- Efficient calculation in time complexity [6 marks]

7.4 Report: 20 marks

1. Correct time complexity given for each algorithm matching code provided [10 marks]
2. Providing correct proofs and explanation [10 marks]

7.5 Mark Deductions

- Does not compile: Mark capped at 50%.
- Code quality – readability, indentation, header comments, method comments, inline comments: Up to −20 marks.
- Instructions not followed (wrong class name, missing readme, something other than a zip submitted, etc): Up to −20 marks
- Late submission: -10% per day

END OF ASSIGNMENT